



GROWMORE GROUP OF INSTITUTE

UNIT – 3

DIMPI GOR

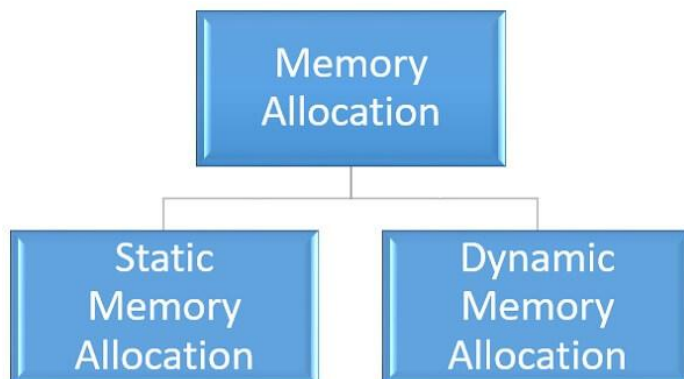
(ASSISTANT PROFESSOR OF GROW MORE BCA & PGDCA)

Memory Allocation

Memory allocation is an action of assigning the physical or the virtual memory address space to a process (its instructions and data). The two fundamental methods of memory allocation are static and dynamic memory allocation.

To get a process executed it must be first placed in the memory. Assigning space to a process in memory is called memory allocation. Memory allocation is a general aspect of the term **binding**.

We have **two** types of memory allocation or we can say two methods of binding, static and dynamic binding.



Types of Memory Allocation

1. Static Memory Allocation

Static memory allocation is performed when the compiler compiles the program and generates object files. The linker merges all these object files and creates a single executable file. The loader loads this single executable file in the main memory, for execution. In static memory allocation, the size of the data required by the process must be known **before** the execution of the process initiates.

If the data sizes are not known before the execution of the process, then they have to be guessed. If the data size guessed is larger than the required, then it leads to **wastage** of memory.

The static memory allocation method does not need any memory allocation operation during the execution of the process. All the memory allocation operation required for the process is done before the execution of the process has started. So, it leads to **faster** execution of a process.

Static memory allocation provides more **efficiency** when compared to dynamic memory allocation.

Advantages of static memory allocation

1. Static memory allocation provides an **efficient** way of assigning the memory to a process.
2. Static memory allocation provides **faster** execution, as at the time of execution it doesn't have to waste time in allocation memory to the program.

Disadvantages of static memory allocation

1. Static memory allocation leads to memory **wastage**. As it estimates the size of memory required by the program. So, if the estimated size is larger, it will lead to memory **wastage** else if the estimated size is smaller, then the program will execute **inappropriately**.

2. Dynamic Memory Allocation

Dynamic memory allocation is performed while the program is in execution. Here, the memory is allocated to the entities of the program when they are to be used for the **first time** while the program is running.

The actual size, of the data required, is known at the run time so, it allocates the **exact** memory space to the program thereby, reducing the memory wastage.

Allocating memory dynamically creates an overhead over the system. Some allocation operations are performed repeatedly during the program execution creating more overheads, leading in **slow** execution of the program.

Dynamic memory allocation does not require special support from the operating system. It is the responsibility of the programmer to design the program in a way to take advantage of dynamic memory allocation method.

Advantages dynamic memory allocation

1. Dynamic memory allocation provides a **flexible** way of assigning the memory to a process.
2. Dynamic memory allocation **reduces** the memory **wastage** as it assigns memory to a process during the execution of that program. So, it is aware of the exact memory size required by the program.

3. If the program is large then the dynamic memory allocation is performed on the different parts of the program. Memory is assigned to the part of a program that is currently in use. This also reduces memory wastage and indeed improves **system performance**.

Disadvantages of dynamic memory allocation

1. Dynamic memory allocation method has an **overhead** of assigning the memory to a process during the time of its execution.
2. Sometimes the memory allocation actions are repeated several times during the execution of the program which leads to more **overheads**.
3. The overheads of memory allocation at the time of its execution **slowdowns** the execution to some extent.

Virtual Memory

Virtual Memory is a storage mechanism which offers user an illusion of having a very big main memory. It is done by treating a part of secondary memory as the main memory. In Virtual memory, the user can store processes with a bigger size than the available main memory.

Why Need Virtual Memory?

- Whenever your computer doesn't have space in the physical memory it writes what it needs to remember to the hard disk in a swap file as virtual memory.

How Virtual Memory Works?

- In the modern world, virtual memory has become quite common these days. It is used whenever some pages require to be loaded in the main memory for the execution, and the memory is not available for those many pages.
- So, in that case, instead of preventing pages from entering in the main memory, the OS searches for the RAM space that are minimum used in the recent times or that are not referenced into the secondary memory to make the space for the new pages in the main memory.

Paging in operating system:

In [computer operating systems](#), **memory paging** is a [memory management](#) scheme by which a computer stores and retrieves data from [secondary storage](#)^[a] for use in [main memory](#).^[citation needed] In this scheme, the operating system retrieves data from secondary storage in same-size [blocks](#) called [pages](#). Paging is an important part of [virtual memory](#) implementations in modern operating systems, using secondary storage to let programs exceed the size of available physical memory.

For simplicity, main memory is called "RAM" (an acronym of [random-access memory](#)) and secondary storage is called "disk" (a shorthand for [hard disk drive](#), [drum memory](#) or [solid-state drive](#), etc.), but as with many aspects of computing, the concepts are independent of the technology used.

Page faults

When a process tries to reference a page not currently present in RAM, the processor treats this invalid memory reference as a [page fault](#) and transfers control from the program to the operating system. The operating system must:

1. Determine the location of the data on disk.
2. Obtain an empty [page frame](#) in RAM to use as a container for the data.
3. Load the requested data into the available page frame.
4. Update the [page table](#) to refer to the new page frame.
5. Return control to the program, transparently retrying the [instruction](#) that caused the page fault.

Demand paging:

In [computer operating systems](#), **demand paging** is a method of [virtual memory](#) management. In a system that uses demand paging, the operating system copies a disk [page](#) into physical memory only if an attempt is made to access it and that page is not already in memory (*i.e.*, if a [page fault](#) occurs). It follows that a [process](#) begins execution with none of its pages in physical memory, and many page faults will occur until most of a process's [working set](#) of pages are located in physical memory. This is an example of a [lazy loading](#) technique.

Demand paging follows that pages should only be brought into memory if the executing process demands them. This is often referred to as [lazy evaluation](#) as only those pages demanded by the process are swapped from [secondary storage](#) to [main memory](#). Contrast this to pure swapping, where all memory for a process is swapped from secondary storage to main memory during the process startup.

Commonly, to achieve this process a [page table](#) implementation is used. The page table maps [logical memory](#) to [physical memory](#). The page table uses a [bitwise](#) operator to mark if a page is valid or invalid. A valid page is one that currently resides in main memory. An invalid page is one that currently resides in secondary memory. When a process tries to access a page, the following steps are generally followed:

- Attempt to access page.
- If page is valid (in memory) then continue processing instruction as normal.
- If page is invalid then a **page-fault trap** occurs.

- Check if the memory reference is a valid reference to a location on secondary memory. If not, the process is terminated (**illegal memory access**). Otherwise, we have to **page in** the required page.
- Schedule disk operation to read the desired page into main memory.
- Restart the instruction that was interrupted by the operating system trap.

Advantages

Demand paging, as opposed to loading all pages immediately:

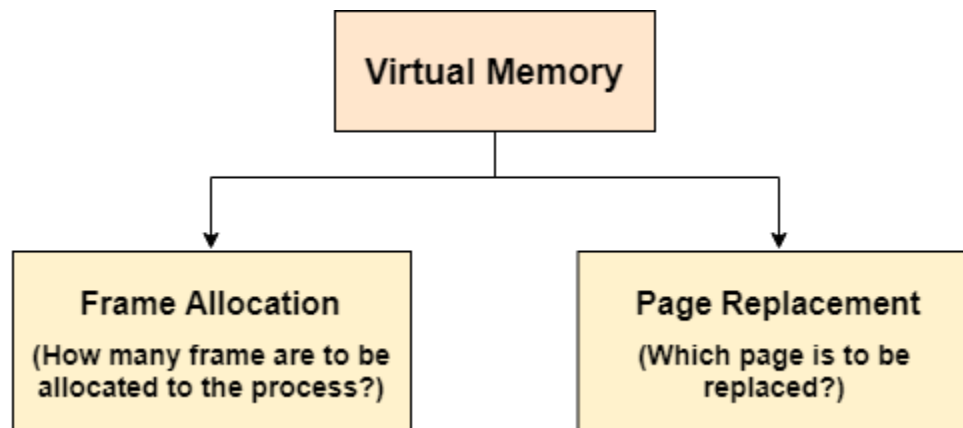
- Only loads pages that are demanded by the executing process.
- As there is more space in main memory, more processes can be loaded, reducing the [context switching](#) time, which utilizes large amounts of resources.
- As main memory is expensive compared to secondary memory, this technique helps significantly reduce the bill of material (BOM) cost in smart phones for example.

Disadvantages

- Low-cost, low-power [embedded systems](#) may not have a [memory management unit](#) that supports page replacement.
- Memory management with [page replacement algorithms](#) becomes slightly more complex.
- Possible security risks, including vulnerability to [timing attacks](#)
- [Thrashing](#) which may occur due to repeated page faults.

Page Replacement Algorithms

The page replacement algorithm decides which memory page is to be replaced. The process of replacement is sometimes called swap out or write to disk. Page replacement is done when the requested page is not found in the main memory (page fault).



There are two main aspects of virtual memory, Frame allocation and Page Replacement. It is very important to have the optimal frame allocation and page replacement algorithm. Frame allocation is all about how many frames are to be allocated to the process while the page replacement is all about determining the page number which needs to be replaced in order to make space for the requested page.

Fragmentation & Defragmentation

Fragmentation is caused when an operating system breaks a file into pieces because there is not enough space on the storage device where the file was originally saved. **Defragmentation** is the term given to the process of scanning the file system and rejoining the split files back into consecutive pieces.

Fragmentation

Fragmentation commonly occurs when old files are opened, modified and subsequently saved.

One example of this, would be where a previously saved file, let's say, a document, is opened and added to. This will cause the file to be larger in physical space than when it was first saved. The operating system will then break the file into 2 or more pieces, and store those pieces (fragments) in different parts of the storage area.

The file system, such as [File Allocation Table \(FAT\)](#) or [NTFS](#), would then keep a record of

where the different fragments of the file are stored.

When the operating system requires the file again, it will query the file system (FAT/NTFS/or other) to find out where the different fragments of the file are located on the partition (drive).

Defragmentation

As noted above, the process of defragmentation rejoins the fragmented parts of a file. It loads the file fragments and then saves them in consecutive parts of the storage.

The process of defragmenting can be time consuming, but it is one of the easiest ways to increase the performance of your computer. The frequency at which a PC should be defragmented will directly depend on the amount of usage area.

Difference between Programmed and Interrupt Initiated I/O

Data transfer between the CPU and I/O devices can be done in variety of modes. These are three possible modes:

1. Programmed I/O
2. Interrupt initiated I/O
3. Direct Memory Access (DMA)

In this article we shall discuss the first **two** modes only.

1.Programmed I/O :

In this mode the data transfer is initiated by the instructions written in a computer program. An input instruction is required to store the data from the device to the CPU and a store instruction is required to transfer the data from the CPU to the device.

Data transfer through this mode requires constant monitoring of the peripheral device by the CPU and also monitor the possibility of new transfer once the transfer has been initiated. Thus CPU stays in a loop until the I/O device indicates that it is ready for data transfer.

Thus programmed I/O is a time consuming process that keeps the processor busy needlessly and leads to wastage of the CPU cycles.

This can be overcome by the use of an interrupt facility. This forms the basis for the Interrupt Initiated I/O.

2. Interrupt Initiated I/O :

This mode uses an interrupt facility and special commands to inform the interface to issue the interrupt command when data becomes available and interface is ready for the data transfer. In the meantime CPU keeps on executing other tasks and need not check for the flag. When the flag is set, the interface is informed and an interrupt is initiated.

This interrupt causes the CPU to deviate from what it is doing to respond to the I/O transfer. The CPU responds to the signal by storing the return address from the program counter (PC) into the memory stack and then branches to service that processes the I/O request. After the transfer is complete, CPU returns to the previous task it was executing.

The branch address of the service can be chosen in two ways known as vectored and non-vectored interrupt. In vectored interrupt, the source that interrupts, supplies the branch information to the CPU while in case of non-vectored interrupt the branch address is assigned to a fixed location in memory.

Difference between Programmed and Interrupt Initiated I/O :

Programmed I/O	Interrupt Initiated I/O
Data transfer is initiated by the means of instructions stored in the computer program. Whenever there is a request for I/O transfer the instructions are executed from the program.	The I/O transfer is initiated by the interrupt command issued to the CPU.
The CPU stays in the loop to know if the device is ready for transfer and has to continuously monitor the peripheral device.	There is no need for the CPU to stay in the loop as the interrupt command interrupts the CPU when the device is ready for data transfer.
This leads to the wastage of CPU cycles as CPU remains busy needlessly and thus the efficiency of system gets reduced.	The CPU cycles are not wasted as CPU continues with other work during this time and hence this method is more efficient.
CPU cannot do any work until the transfer is complete as it has to stay in the loop to continuously monitor the peripheral device.	CPU can do any other work until it is interrupted by the command indicating the readiness of device for data transfer
Its module is treated as a slow module.	Its module is faster than programmed I/O module.
It is quite easy to program and understand.	It can be tricky and complicated to understand if one uses low level language.
The performance of the system is severely degraded.	The performance of the system is enhanced to some extent.